BattleGrid

Software Testing Document

Serkan EROL 150122619
Yusuf Emir ONAN 524125010
Başak AKYÜZ 150122827

29.05.2026
CSE3044 Software Engineering Term Project

Table of Contents

1. Introduction

This document describes the testing activities conducted for the BattleGrid system, an online turn-based Battleship game implemented as a client-server web application. It covers the testing methods used, the test cases executed, defects identified during testing, and the final test results.

The automated test suite is located in the BattleGrid.Tests project and is built on the xUnit testing framework. It includes unit tests for isolated Domain/GameLogic and leaderboard logic, integration tests using ASP.NET Core WebApplicationFactory, and response-time tests executed against the API and PostgreSQL database.

2. Testing Methods

2.1 Unit Testing (Domain/GameLogic)

Unit tests validate the core game logic in isolation, without any database or HTTP involvement. They target the BattleGrid.Domain.GameLogic package and cover the following classes:

- CoordinateTests: Verifies that valid coordinates are created correctly and out-of-bounds coordinates throw exceptions.
- BoardTests: Covers hit registration, miss registration, detection of already-targeted cells, overlapping ship prevention, and correct ship identification on hit.
- GameStateTests: Validates game phase transitions, turn management (Player 1 starts, turns alternate on miss, do not switch on hit), invalid player action rejection, and win condition detection.
- ShipPlacementHelperTests: Confirms that horizontal and vertical ship coordinate generation produces correct results.

A static TestShipFactory helper class generates reusable, predefined ship coordinate sets so that placement scenarios do not need to repeat coordinate construction logic across tests.

LeaderboardAllTimeBuilderTests and LeaderboardWindowBuilderTests validate leaderboard construction, ordering, viewer context windows, all-time leaderboard aggregation, and duplicate rank handling without HTTP or database dependencies.

2.2 Integration Testing (API Endpoints)

Integration tests exercise the full HTTP pipeline from controller through application services to the PostgreSQL database. They use xUnit's IAsyncLifetime interface for per-test setup and teardown, and ASP.NET Core's WebApplicationFactory<Program> to host the real API in-process.

Key infrastructure elements:

- BattleGridApiFactory: Instantiates the API in the Development environment, removes all background IHostedService registrations (matchmaking, ban cleanup) to avoid interference, and exposes a CanConnectToDatabaseAsync() probe so tests skip gracefully when PostgreSQL is unavailable.

- IntegrationTestCollection: Declares a single xUnit collection fixture with DisableParallelization = true, ensuring all tests share one WebApplicationFactory instance and execute serially to prevent race conditions on shared database state.
- IntegrationApiTestBase: Base class providing a per-test HttpClient, DatabaseAvailable flag, and async teardown via IntegrationTestDataTracker.
- IntegrationTestDataTracker: Tracks emails and match IDs created during each test and calls CleanupAllAsync on disposal, guaranteeing database rows are removed after every test regardless of pass or fail.
- ApiIntegrationTestHelper: Static helper providing factory methods for creating unique register requests, posting to auth endpoints, seeding database fixtures (matches, placements, moves, leaderboard rows), creating authenticated HTTP clients, and cleaning up test users.

Test classes organized by API area: AuthEndpointTests, UserEndpointTests, AdminEndpointTests, MatchEndpointTests, LeaderboardEndpointTests, ShipTypeEndpointTests, ShipPlacementEndpointTests, HealthEndpointTests, and AuthResponseTimeTests.

2.3 Performance / Response-Time Testing

AuthResponseTimeTests measures the round-trip time for authentication-related operations as experienced by the Blazor UI: the timer starts immediately before the HTTP POST is issued and stops when the response is received. This replicates the latency the user observes on form submit.

Two types of performance measurements are taken:

- Single-run measurements: one timed request per test, result logged via ITestOutputHelper.
- Multi-run averages: 5 sequential iterations of register and login; minimum, maximum, and average elapsed milliseconds are reported. These tests do not assert a hard latency threshold but produce benchmark data for comparison against the SRS requirement of 2-second response time for web actions.

All performance tests also assert correctness (correct HTTP status code, DB persistence), so they serve as both timing benchmarks and functional regression guards.

2.4 Manual Testing

In addition to automated tests, manual testing was conducted throughout development via two channels:

- SwaggerUI: Used to test API endpoints interactively, inspecting request/response bodies and headers during early development and after major changes.
- BattleGrid.Console: A standalone console application that instantiates the Domain game logic directly (no API, no DB) to allow developers to simulate a full two-player Battleship game from the command line. This was the primary validation tool for game logic before unit tests were written.

3. Test Cases

The automated test cases are listed below, organized by test level and API area. Each entry identifies the test case ID, a short description, the tested input or precondition, the expected result, and the execution status.

Legend for Status column:

- PASS: Test executed and all assertions passed.
- FAIL: Test executed and at least one assertion failed (see Section 4 for details).
- SKIP: DatabaseAvailable was false; test was bypassed gracefully.

3.1 Unit Test Cases

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-UNIT-01	Create valid coordinate	Valid X and Y values within the board boundary	Coordinate object is created successfully	**PASS**
TC-UNIT-02	Reject coordinate with negative X	X = -1, Y = 0	Exception is thrown for invalid coordinates	**PASS**
TC-UNIT-03	Reject coordinate with negative Y	X = 0, Y = -1	Exception is thrown for invalid coordinates	**PASS**
TC-UNIT-04	Reject coordinate with X outside upper boundary	X = 10, Y = 0	Exception is thrown for invalid coordinates	**PASS**
TC-UNIT-05	Reject coordinate with Y outside upper boundary	X = 0, Y = 10	Exception is thrown for invalid coordinates	**PASS**
TC-UNIT-06	Register miss on board	Attack coordinate without a ship	Cell is marked as Miss	**PASS**
TC-UNIT-07	Register hit on board	Attack coordinate containing a ship	Cell is marked as Hit	**PASS**
TC-UNIT-08	Detect already-targeted cell	Attack the same coordinate more than once	AlreadyTargeted result is returned	**PASS**
TC-UNIT-09	Prevent overlapping ship placement	Attempt to place ships on overlapping coordinates	Overlapping placement is rejected	**PASS**
TC-UNIT-10	Detect hit on correct ship	Attack a coordinate belonging to a specific placed ship	Correct ship is identified as hit	**PASS**
TC-UNIT-11	Handle multiple misses	Multiple attacks on empty coordinates	All empty attacks are registered as misses	**PASS**

TC-UNIT-12	Initial game phase	New GameState instance	Initial game phase is ShipPlacement	**PASS**
TC-UNIT-13	Start game after ship placement	Both players complete required ship placements	Game phase changes to InProgress	**PASS**
TC-UNIT-14	Player 1 starts first turn	Game enters active gameplay	Player 1 is selected as the starting player	**PASS**
TC-UNIT-15	Switch turn after miss	Current player fires at an empty coordinate	Turn switches to the opponent	**PASS**
TC-UNIT-16	Alternate turns on repeated misses	Players repeatedly miss their shots	Turn alternates correctly between players	**PASS**
TC-UNIT-17	Keep turn after hit	Current player fires at a ship coordinate	Current player keeps the turn after a hit	**PASS**
TC-UNIT-18	Reject wrong-player shot	Player who does not own the current turn attempts to shoot	Invalid action is rejected with an exception	**PASS**
TC-UNIT-19	Preserve winner on winning shot	Winning shot is fired	Winner remains assigned and turn is not switched incorrectly	**PASS**
TC-UNIT-20	End game when player wins	All opponent ships are destroyed	Game ends and winner is detected	**PASS**
TC-UNIT-21	Generate horizontal ship coordinates	Ship start coordinate, length, and horizontal orientation	Expected horizontal coordinate list is generated	**PASS**
TC-UNIT-22	Generate vertical ship coordinates	Ship start coordinate, length, and vertical orientation	Expected vertical coordinate list is generated	**PASS**
TC-UNIT-23	Generate horizontal rectangle placement	Rectangle start coordinate and horizontal dimensions	Expected rectangle coordinates are generated	**PASS**
TC-UNIT-24	Generate vertical rectangle placement	Rectangle generation with vertical orientation	Width/height span is handled correctly for vertical placement	**PASS**
TC-UNIT-25	Reject out-of-bounds rectangle placement	Rectangle generation that exceeds board boundary	Generation fails and returns false	**PASS**

TC-UNIT-26	Use highest rating for all-time entry	Player statistics with current and highest rating values	All-time entry uses highest rating correctly	**PASS**
TC-UNIT-27	Order all-time leaderboard by peak rating	Multiple players with different peak ratings	All-time leaderboard is ordered by peak highest rating	**PASS**
TC-UNIT-28	Return top 100 for anonymous viewer	Anonymous viewer and leaderboard with more than 100 entries	Only top 100 entries are returned	**PASS**
TC-UNIT-29	Return top 100 for viewer inside top 100	Authenticated viewer ranked inside top 100	Top 100 entries are returned without extra context rows	**PASS**
TC-UNIT-30	Include context for viewer outside top 100	Authenticated viewer ranked outside top 100	Viewer row and nearby context rows are included	**PASS**
TC-UNIT-31	Include context near leaderboard bottom	Viewer ranked near the bottom of the leaderboard	Available rows below viewer are included correctly	**PASS**
TC-UNIT-32	Handle viewer at final rank	Viewer ranked at the final leaderboard position	Context window is generated without out-of-range rows	**PASS**
TC-UNIT-33	Deduplicate overlapping leaderboard ranks	Top list and viewer context list with overlapping ranks	Duplicate ranks are removed from merged display list	**PASS**

3.2 Health Endpoint Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-HEALTH-01	Root endpoint	GET /	Response body contains 'BattleGrid API is running'	**PASS**
TC-HEALTH-02	Health endpoint	GET /health	HTTP 200, Status='ok', Database='connected'	**PASS**

3.3 Authentication Endpoint Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-AUTH-01	Register with valid payload	POST /api/Auth/register with valid unique credentials	HTTP 200, success message	**PASS**
TC-AUTH-02	Register with duplicate email	POST /api/Auth/register with	HTTP 400 Bad Request	**PASS**

		already-registered email		
TC-AUTH-03	Register with duplicate username	POST /api/Auth/register with existing username, new email	HTTP 400 Bad Request	**PASS**
TC-AUTH-04	Register with mismatched passwords	POST /api/Auth/register with Password != ConfirmPassword	HTTP 400 Bad Request	**PASS**
TC-AUTH-05	Login with valid credentials	POST /api/Auth/login with correct email and password	HTTP 200, AccessToken and RefreshToken returned	**PASS**
TC-AUTH-06	Login with wrong password	POST /api/Auth/login with registered email, wrong password	HTTP 401 Unauthorized	**PASS**
TC-AUTH-07	Login with non-existing email	POST /api/Auth/login with unregistered email	HTTP 401 Unauthorized	**PASS**
TC-AUTH-08	Login with non-existing username	POST /api/Auth/login with unregistered username	HTTP 401 Unauthorized	**PASS**
TC-AUTH-09	Logout with valid refresh token	POST /api/Auth/logout with valid RefreshToken	HTTP 200 OK	**PASS**
TC-AUTH-10	Token refresh with valid token	POST /api/Auth/refresh with valid RefreshToken	HTTP 200, new AccessToken returned	**PASS**
TC-AUTH-11	Update password without auth	PATCH /api/Auth/password with no Bearer token	HTTP 401 Unauthorized	**PASS**
TC-AUTH-12	Update password with auth - success	PATCH /api/Auth/password with valid token and correct old password	HTTP 200, re-login with new password succeeds	**PASS**
TC-AUTH-13	Update password with mismatched confirmation	PATCH /api/Auth/password with NewPassword != ConfirmNewPassword	HTTP 400 Bad Request	**PASS**
TC-AUTH-14	Update password same old and new	PATCH /api/Auth/password where OldPassword == NewPassword	HTTP 400 Bad Request	**PASS**

3.4 User Endpoint Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-USER-01	Get user by email (no auth)	GET /api/User/{email} for registered user	HTTP 200, matching user data	**PASS**
TC-USER-02	Get unknown user	GET /api/User/{email} for non-existent user	HTTP 404 Not Found	**PASS**
TC-USER-03	Get current user (me) with auth	GET /api/User/me with valid Bearer token	HTTP 200, authenticated user's data	**PASS**
TC-USER-04	Get current user without auth	GET /api/User/me with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-05	Get user by ID without auth	GET /api/User/{id} with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-06	Get user by ID with auth	GET /api/User/{id} with valid token	HTTP 200, correct user returned	**PASS**
TC-USER-07	Get all users as non-admin	GET /api/User/all with regular user token	HTTP 403 Forbidden	**PASS**
TC-USER-08	Get all users without auth	GET /api/User/all with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-09	Get all users as admin	GET /api/User/all?page=1&pageSize=20 with admin token	HTTP 200, paginated user list	**PASS**
TC-USER-10	Get ban status with auth	GET /api/User/ban-status with valid token	HTTP 200, IsBanned=false for new user	**PASS**
TC-USER-11	Get ban status without auth	GET /api/User/ban-status with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-12	Get current season stats with auth	GET /api/User/stats/current-season with valid token	HTTP 200, valid season stats	**PASS**
TC-USER-13	Get current season stats without auth	GET /api/User/stats/current-season with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-14	Change email without auth	PATCH /api/User/email with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-15	Change username without auth	PATCH /api/User/username with no token	HTTP 401 Unauthorized	**PASS**
TC-USER-16	Deactivate account without auth	PATCH /api/User/deactivate with no token	HTTP 401 Unauthorized	**PASS**

3.5 Admin Endpoint Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-ADMIN-01	Ban player as non-admin	POST /api/Admin/players/ban with regular user token	HTTP 403 Forbidden	**PASS**
TC-ADMIN-02	Ban player without auth	POST /api/Admin/players/ban with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-03	Unban player as non-admin	POST /api/Admin/players/unban with regular user token	HTTP 403 Forbidden	**PASS**
TC-ADMIN-04	Unban player without auth	POST /api/Admin/players/unban with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-05	Grant admin as non-admin	POST /api/Admin/players/grant-admin with regular token	HTTP 403 Forbidden	**PASS**
TC-ADMIN-06	Grant admin without auth	POST /api/Admin/players/grant-admin with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-07	Grant admin as admin - success	POST /api/Admin/players/grant-admin with admin token	HTTP 200, user IsAdmin=true in DB	**PASS**
TC-ADMIN-08	Grant admin for unknown player	POST /api/Admin/players/grant-admin with non-existent email	HTTP 400 Bad Request	**PASS**
TC-ADMIN-09	Ban and unban player as admin	Full ban then unban cycle with admin token	Both HTTP 200 OK	**PASS**
TC-ADMIN-10	Ban/unban unknown player as admin	Ban/unban with non-existent email via admin token	HTTP 400 Bad Request for both	**PASS**
TC-ADMIN-11	Get player profile as admin	GET /api/Admin/players/{id}/profile with admin token	HTTP 200, correct user data	**PASS**
TC-ADMIN-12	Get player profile without auth	GET /api/Admin/players/1/profile with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-13	Get player profile as non-admin	GET /api/Admin/players/1/profile with regular token	HTTP 403 Forbidden	**PASS**
TC-ADMIN-14	Get player profile for unknown player	GET /api/Admin/players/999999/profile with admin token	HTTP 404 Not Found	**PASS**
TC-	Advance season	POST /api/Admin/season/advance	HTTP 200, new	**PASS**

ADMIN-15	as admin	with admin token	season row created in DB	
TC-ADMIN-16	Advance season as non-admin	POST /api/Admin/season/advance with regular token	HTTP 403 Forbidden, no DB changes	**PASS**
TC-ADMIN-17	Advance season without auth	POST /api/Admin/season/advance with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-18	Get player match history as admin	GET /api/Admin/players/{id}/match-history with admin token	HTTP 200, valid history list	**PASS**
TC-ADMIN-19	Get player match history without auth	GET /api/Admin/players/1/match-history with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-20	Get player match history as non-admin	GET /api/Admin/players/1/match-history with regular token	HTTP 403 Forbidden	**PASS**
TC-ADMIN-21	Get player match history for unknown player	GET /api/Admin/players/999999/match-history with admin token	HTTP 404 Not Found	**PASS**
TC-ADMIN-22	Get admin match replay without auth	GET /api/Admin/matches/999999/replay with no token	HTTP 401 Unauthorized	**PASS**
TC-ADMIN-23	Get admin match replay as non-admin	GET /api/Admin/matches/999999/replay with regular token	HTTP 403 Forbidden	**PASS**
TC-ADMIN-24	Get admin match replay as admin - not found	GET /api/Admin/matches/999999/replay with admin token	HTTP 404 Not Found	**PASS**
TC-ADMIN-25	Get admin match replay as admin - success	GET /api/Admin/matches/{id}/replay, fixture seeded	HTTP 200, full replay data	**PASS**

3.6 Match Endpoint Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-MATCH-01	Get resumable match with auth (no active match)	GET /api/Match/resumable, user has no active match	HTTP 204 No Content	**PASS**
TC-MATCH-02	Get resumable match with auth (active match exists)	GET /api/Match/resumable, match fixture seeded in DB	HTTP 200, correct MatchId returned	**PASS**
TC-MATCH-03	Get resumable match without auth	GET /api/Match/resumable with no token	HTTP 401 Unauthorized	**PASS**
TC-	Get match history	GET	HTTP 200, non-	**PASS**

MATCH-04	with auth	/api/Match/history?limit=10 with valid token	null Matches list	
TC-MATCH-05	Get match history without auth	GET /api/Match/history?limit=10 with no token	HTTP 401 Unauthorized	**PASS**
TC-MATCH-06	Get replay for unknown match	GET /api/Match/999999/replay with valid token	HTTP 404 Not Found	**PASS**
TC-MATCH-07	Get replay for valid match with auth	GET /api/Match/{id}/replay, replay fixture seeded	HTTP 200, replay with moves and placements	**PASS**
TC-MATCH-08	Get replay without auth	GET /api/Match/999999/replay with no token	HTTP 401 Unauthorized	**PASS**
TC-MATCH-09	Get recovery for unknown match	GET /api/Match/999999/recovery with valid token	HTTP 204 No Content	**PASS**
TC-MATCH-10	Get recovery with auth (valid match)	GET /api/Match/{id}/recovery, terminal match seeded	HTTP 200, correct status and player IDs	**PASS**
TC-MATCH-11	Get recovery without auth	GET /api/Match/999999/recovery with no token	HTTP 401 Unauthorized	**PASS**

3.7 Leaderboard Endpoint Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-LB-01	Get current season leaderboard	GET /api/Leaderboard/season (unauthenticated)	HTTP 200, valid entries with correct fields	**PASS**
TC-LB-02	Get all-time leaderboard	GET /api/Leaderboard/all-time (unauthenticated)	HTTP 200, valid entries	**PASS**
TC-LB-03	Viewer context outside top 100	GET /api/Leaderboard/season as user ranked >100, 115 users seeded	HTTP 200, viewer entry + context rows included	**PASS**

3.8 Ship Type and Ship Placement Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-SHIP-01	Get all ship types	GET /api/ShipType/all (unauthenticated)	HTTP 200, non-empty list with valid ShipName, Length,	**PASS**

			MaxPerPlayer	
TC-SHIP-02	Place ship without auth	POST /api/ShipPlacement/placeShip with no token	HTTP 401 Unauthorized	**PASS**
TC-SHIP-03	Place ship - invalid match	POST /api/ShipPlacement/placeShip with valid token, MatchID=999999	HTTP 401 Unauthorized	**PASS**

3.9 Authentication Response-Time Tests

Test ID	Test Case	Input / Precondition	Expected Result	Status
TC-PERF-01	Register response time measurement	Measure elapsed ms from POST /api/Auth/register submit to 200 response; verify DB persistence	HTTP 200, user in DB, timing logged	**PASS**
TC-PERF-02	Register mismatched passwords - timing	Measure elapsed ms for 400 on mismatched password register	HTTP 400, timing logged	**PASS**
TC-PERF-03	Login response time measurement	Measure elapsed ms from POST /api/Auth/login to 200; verify session row in DB	HTTP 200, session persisted, timing logged	**PASS**
TC-PERF-04	Register - average over 5 runs	Execute 5 sequential register requests, collect all timings	All 200 OK, average > 0 ms, stats logged	**PASS**
TC-PERF-05	Login - average over 5 runs	Execute 5 sequential login cycles, collect all timings	All 200 OK, average > 0 ms, stats logged	**PASS**
TC-PERF-06	Logout response time measurement	Measure elapsed ms for POST /api/Auth/logout	HTTP 200, timing logged	**PASS**
TC-PERF-07	Register duplicate email - timing	Measure elapsed ms for 400 on duplicate email	HTTP 400, timing logged	**PASS**
TC-PERF-08	Login wrong password - timing	Measure elapsed ms for 401 on wrong password	HTTP 401, timing logged	**PASS**
TC-PERF-09	Login non-existing user - timing	Measure elapsed ms for 401 on missing user	HTTP 401, timing logged	**PASS**

TC-PERF-10	Change password success - timing	Measure elapsed ms for successful PATCH /api/Auth/password	HTTP 200, timing logged	**PASS**
TC-PERF-11	Change password wrong old - timing	Measure elapsed ms for failed PATCH /api/Auth/password (wrong old)	HTTP 400, timing logged	**PASS**
TC-PERF-12	Change password mismatch - timing	Measure elapsed ms for failed PATCH (mismatched confirmation)	HTTP 400, timing logged	**PASS**
TC-PERF-13	Change password same old/new - timing	Measure elapsed ms for failed PATCH (same old and new)	HTTP 400, timing logged	**PASS**
TC-PERF-14	Change email success - timing	Measure elapsed ms for successful PATCH /api/User/email	HTTP 200, timing logged	**PASS**
TC-PERF-15	Change email wrong password - timing	Measure elapsed ms for failed PATCH /api/User/email	HTTP 400, timing logged	**PASS**
TC-PERF-16	Change username success - timing	Measure elapsed ms for successful PATCH /api/User/username	HTTP 200, timing logged	**PASS**
TC-PERF-17	Change username wrong password - timing	Measure elapsed ms for failed PATCH /api/User/username	HTTP 400, timing logged	**PASS**

4. Errors and Bugs Identified

The following issues were discovered during the testing process. Each entry records the bug, the test or observation that revealed it, the root cause, and how it was resolved.

ID	Bug Description	Discovered By	Root Cause	Resolution
BUG-01	Register success message contained a typo: 'succesfully' instead of 'successfully'	TC-AUTH-01 assertion comparing response body against ExpectedRegisterSuccessMessage	Typo in the string literal returned by AuthController.Register on success	The typo is corrected.
BUG-02	PlaceShip with an invalid MatchID returned 401 Unauthorized instead of 404	TC-SHIP-03 observed the 401 response when MatchID=999999 was submitted with	ShipPlacementController verifies the player is a participant in the requested match	Behaviour confirmed intentional by the team: if a match does not exist, the player has no valid claim to

	Not Found	a valid token	before returning a placement result. A non-existent match produces no participant record, which was treated identically to an unauthorized player.	it, so 401 is an acceptable and consistent security response. Test expectation updated to assert 401.
BUG-03	Leaderboard viewer-context test failed intermittently due to season number collision	TC-LB-03 occasionally failed when a previous test run left PlayerStat rows behind, causing GetNextGlobalSeasonNoAsync to return a season already containing seeded rows	CleanupOrphanedIntegrationUsersAsync ran at the end of a test, but if the test process was killed mid-run, old rows persisted and polluted the next run's season calculation	Added a dedicated CleanupOrphanedIntegrationUsersAsync call at the start of the leaderboard viewer-context test. Also ensured DeletePlayerStatSeasonAsync is called for all seeded rows during teardown.
BUG-04	AdvanceSeason created a duplicate PlayerStat row when the admin already had a row for the new season	TC-ADMIN-15 (AdvanceSeason_AsAdmin_ReturnsOk_AndCreatesSeasonRow) asserted a DB row was created. On the second run of the test in the same session the assertion passed but an extra duplicate row existed in PlayerStat.	The AdvanceSeason service did not check for an existing row before inserting. The teardown deleted only the newly reported NewSeasonNo row, leaving residuals from prior runs.	Service was patched to use an INSERT … ON CONFLICT DO UPDATE pattern. Teardown now calls DeletePlayerStatSeasonAsync for the returned NewSeasonNo, and CleanupOrphanedIntegrationUsersAsync handles any remaining orphans.
BUG-05	GetRecovery returned 200 with null WinnerUserId for a P1Won match, failing the assertion	TC-MATCH-10 asserted recovery.WinnerUserId == user.Profile.UserID but received null	CreateRecoveryMatchFixtureAsync did not set a WinnerUserId column; the match entity had no explicit winner field at the time.	A WinnerUserId property was added to the Match entity and set in the recovery fixture helper based on the terminal status (P1Won maps to Player1ID). The MatchRecoveryStateDto was updated to expose this field.
BUG-06	Leaderboard viewer-context	TC-UNIT-31 and TC-UNIT-32; the	LeaderboardWindowBuilder	LeaderboardWindowBuilder was fixed to

	window threw an unhandled exception when the authenticated viewer was the last-ranked player in the season	endpoint returned a 500 Internal Server Error instead of a valid leaderboard response when the viewer held the lowest rank	unconditionally attempted to fetch context rows below the viewer's rank. When the viewer was the final entry, the below-context query returned an empty collection and the builder tried to access an element that did not exist, throwing an InvalidOperationException	guard against an empty below-context result.

No critical security vulnerabilities or data-loss defects were identified during the testing period. All six bugs listed above were resolved before final test execution.

5. Test Results

5.1 Summary

Test Area	Total Tests	Passed
Domain / Game Logic Unit Tests	25	25
Leaderboard Unit Tests	8	8
Health Endpoint	2	2
Authentication Endpoints	14	14
User Endpoints	16	16
Admin Endpoints	25	25
Match Endpoints	11	11
Leaderboard Endpoints	3	3
Ship Type / Placement	3	3
Auth Response-Time Tests	17	17
Total	**124**	**124**

5.2 Coverage Observations

The complete automated test suite includes 124 passing tests: 33 unit tests and 91 integration/performance tests. The unit tests cover isolated Domain/GameLogic and leaderboard logic, while the integration test suite achieves broad endpoint coverage across the REST API surface.

- All authentication flows: registration, login, logout, token refresh, and password update.
- Role-based access control: every admin-only endpoint is tested for Unauthorized (no token), Forbidden (regular user), and OK (admin) responses.
- Match lifecycle read paths: resumable match detection, match history, replay retrieval, and recovery state.

- Leaderboard pagination and viewer rank context logic.
- Ship type catalogue and ship placement authorization.
- System health and root endpoint.

The following areas are outside the current integration test scope and represent future testing opportunities:

- Real-time SignalR hub interactions (matchmaking queue join/leave, in-game move events). These require WebSocket test clients and are not covered by the HTTP integration suite.
- Full end-to-end game play (placing ships and firing shots through the hub) was validated manually via the BattleGrid.Console tool and SwaggerUI but not automated.
- Concurrency scenarios (two clients simultaneously joining the matchmaking queue) were not systematically tested due to the DisableParallelization constraint on the integration test collection.

5.3 Performance Observations

Response-time tests (TC-PERF-01 through TC-PERF-17) were executed against a local development PostgreSQL instance. Representative timings observed:

- Register (single run): typically 80-220 ms, within the 2-second SRS requirement.
- Login (single run): typically 60-180 ms, within the 2-second SRS requirement.
- Logout: typically 20-60 ms.
- Password/email/username change: typically 50-150 ms.
- Register average over 5 runs: approximately 100-160 ms.
- Login average over 5 runs: approximately 70-130 ms.

All measured times are well within the 2-second threshold specified in the DSD. Background IHostedService registrations (matchmaking, ban cleanup) are removed during testing via BattleGridApiFactory to prevent interference with timing measurements.

5.4 Conclusion

All 124 automated tests pass after the five defects documented in Section 4 were resolved or accepted as intentional behavior. This total includes 33 unit tests and 91 integration/performance tests. The system's REST API correctly enforces authentication and authorization policies, returns appropriate HTTP status codes for valid and invalid inputs, and persists and retrieves data as specified in the Design Specification Document. Response times for authentication operations are well within the 2-second target defined by the SRS. The test infrastructure provides reliable automated regression coverage for current API endpoints and core game logic.

6. References

- CSE3044 Software Engineering Course Project Guideline Slides
- CSE3044 Software Engineering Course Lecture Slides
- BattleGrid Initial Report
- BattleGrid Requirement Specification Document

- BattleGrid Desing Specification Document
- IEEE Std 829-1998: IEEE Standard for Software Test Documentation framework guidelines